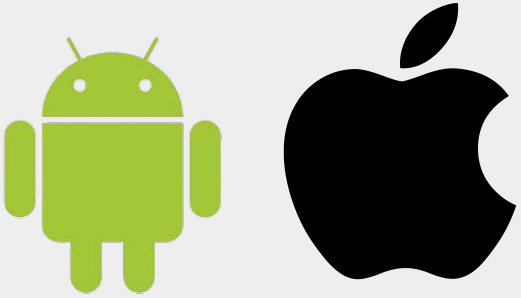
Kotlin Multiplatform

Android Lecture 6

Simona Kurňavová

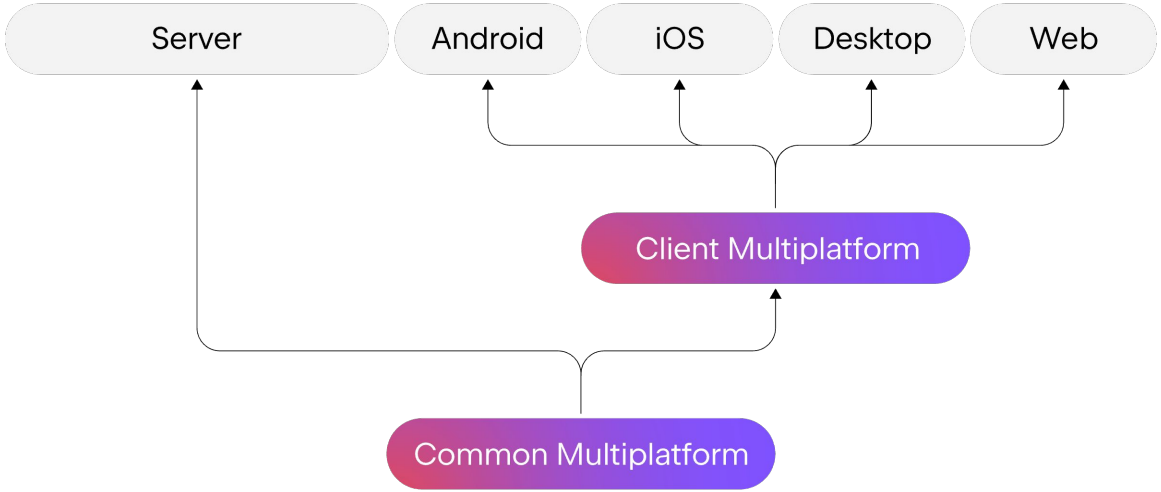
Today’s Agenda

- Multiplatform development Overview
- Kotlin Multiplatform
- iOS development
- Q&A



Multiplatform development

Multiplatform development

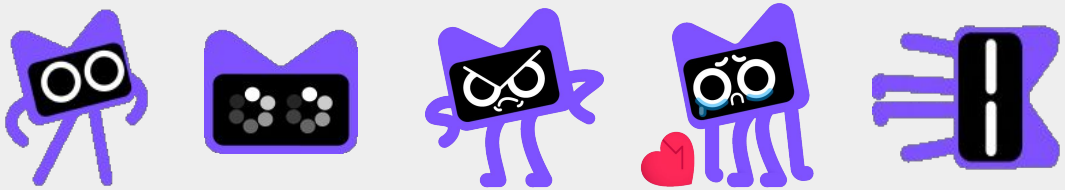


Benefits of multiplatform development

- Reusable code
- Time savings
- Effective resource management
- Attractive opportunities for developers
- Opportunity to reach wider audiences
- Quicker time to market and customization

Technologies

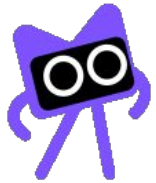
- Flutter (Dart)
  - Flutter flow
  - Supports: mobile, desktop, web, embedded
- React Native (JavaScript)
  - Supports: mobile, desktop, Apple's visionOS, tv, web
- Xamarin (C++)
  - Support ended in May 2024
  - Replaced with .NET MAUI
    - Supports: mobile, desktop, Tizen (Samsung wearables)
- Kotlin multiplatform (Kotlin + native languages)



Kotlin Multiplatform

Kotlin assets

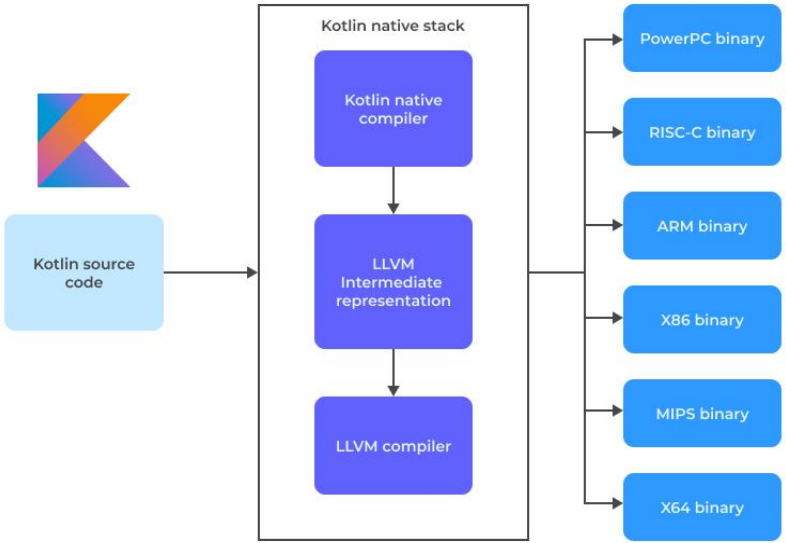
Kotlin Multiplatform (KMP)



- **Documentation**
- JetBrains
- Announced in 2017
- Cross-platform development
- Allows to write shared business logic in Kotlin and use it across multiple platforms
- IDEs support
- Built on Kotlin/Native - technology and compiler that allows Kotlin code to be compiled directly into native binaries for various platforms, without relying on a virtual machine
- Primary used for iOS and Android (KMM)
- Kotlin Multiplatform Gradle plugin

Kotlin/Native

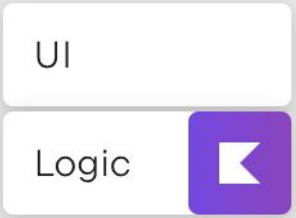
- Kotlin/Native is a technology for compiling Kotlin code to native binaries which can run without a virtual machine.
- [mDevCamp talk about Kotlin/Native] from Filip Dolník (Touchlab)



Different approaches to code sharing with KMP

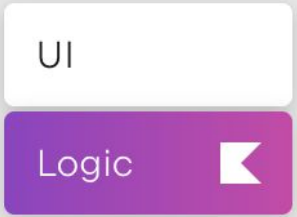
Share a piece of logic

Improve your app's stability by sharing an isolated and critical part of the app. Reuse the Kotlin code you already have to keep the applications in sync.



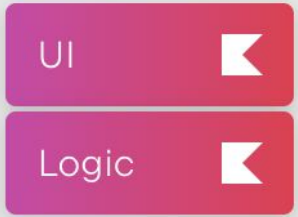
Share logic and keep the UI native

Use Kotlin Multiplatform when you start a new project, and implement data handling and business logic just once. Keep the UI native to meet the most stringent requirements.



Share up to 100% of the code

Elevate development efficiency and share up to 100% of your code with Compose Multiplatform – a modern declarative framework by JetBrains for sharing UI across multiple platforms.



Kotlin Multiplatform

- **Pros**:
  - Reduced duplicity of code
  - Reduced development time
  - Consistent architecture
  - Shared libraries
  - Native code for UI for both platforms (better UX)

- **Cons**:
  - Platform specific code often needed
  - Complex project (bigger repository, bigger dev team, …)
  - Not all libraries/tools support KMM
  - Switching between two IDEs

iOS development

Apple iOS – Version summary

- iOS 1 (2007): original iOS with first iPhone, Safari, apps primary web-based, third-party development not supported
- iOS 2 (2008): App Store
- iOS 3 (2009): Copy-Paste, Spotlight Search.
- iOS 4 (2010): Multitasking, FaceTime.
- iOS 5 (2011): Notification Center, iMessage.
- iOS 6 (2012): Apple Maps, Passbook.
- iOS 7 (2013): Flat design, Control Center.
- iOS 8 (2014): HealthKit, HomeKit, Extensions.
- iOS 9 (2015): Split View on iPad, Siri improvements.
- iOS 10 (2016): Widgets, iMessage apps.
- iOS 11 (2017): Files app, ARKit.
- iOS 12 (2018): Performance improvements, Screen Time.
- iOS 13 (2019): Dark Mode, Sign In with Apple.
- iOS 14 (2020): Widgets, App Library, App Clips.
- iOS 15 (2021): Changes to Apple maps.
- iOS 16 (2022): Redesigned lock screen.
- iOS 17 (2023): StandBy, Interactive widgets.
- iOS 18 (2024): Apple Intelligence, Message scheduling

iOS development

- **Swift**: introduced in 2014
- **Objective-C**: used in old apps, new apps should use Swift
- **Xcode IDE**: editor, simulation, debugging
- **UIKit, SwiftUI**
- Apps distributed through the Apple App Store
- Apple Developer Program
- Development possible only on MacOS



iOS vs. Android comparison

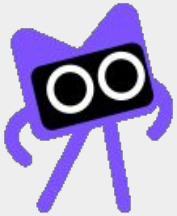
	iOS	Android
Fragmentation	One manufacturer	Multiple manufacturers and range of devices
App distribution	App store	Google play, third party stores
Updates	Consistent across all devices	Often delays due to updates going through different manufacturers
Technology	Swift, Objective-C, UIKit, SwiftUI	Kotlin, Java, XML, Compose
Development environment	Xcode (on MacOS only)	Android Studio (Windows, MacOS, Linux), InteliJ, Eclipse, Fleet

Swift

- **Documentation:** https://developer.apple.com/swift/
- Developed by Apple for iOS, macOS, watchOS, and tvOS
- Open source
- Swift playground (iPad, Mac)
- Xcode support

Swift

- Interoperable with Objective-C
- Comparable speed to low-level languages.
- Automatic Reference Counting (ARC) for efficient memory management.
- Typesafe: optionals, type inference, and static typing
- Robust error-handling model

Swift vs. Kotlin

Swift vs. Kotlin: Variables, conditions, loops

SWIFT

```swift
var optionalVariable: Int? = 42
let myConstant: String = "Swift"

if condition {
    // Code for true condition
} else {
    // Code for false condition
}

for item in array {
    // Code
}
```

KOTLIN

```kotlin
var optionalVariable: Int? = 42
val myConstant: String = "Kotlin"

if (condition) {
    // Code for true condition
} else {
    // Code for false condition
}

for (item in array) {
    // Code for each item
}
```

Swift vs. Kotlin: Functions and classes

SWIFT

```swift
func greet(name: String = "Alice") -> String {
    return "Hello, \(name)!" // Interpolation
}

class Person {
    var name: String

    init(name: String) {
        self.name = name
    }
}

struct Point {
    var x, y: Double
}
```

KOTLIN

```kotlin
fun greet(name: String = "Alice"): String {
    return "Hello, $name!" // Interpolation
}

fun greet2(name: String = "Alice"): String =
    "Hello, $name!"

class Person(
    var name: String
)

data class Point(
    var x: Double,
    var y: Double
)
```

Swift vs. Kotlin: Enums

SWIFT

```swift
enum Color {
    case red
    case green
    case blue
}

let color: Color = .red

switch color {
case .red:
    print("It's red!")
case .green:
    print("It's green!")
case .blue:
    print("It's blue!")
}
```

KOTLIN

```kotlin
enum class Color {
    RED, GREEN, BLUE
}

val color = Color.RED

when (color) {
    Color.RED -> println("It's red!")
    Color.GREEN -> println("It's green!")
    Color.BLUE -> println("It's blue!")
}
```

Swift vs. Kotlin: Null-safety

SWIFT

```swift
let length: Int? = nullableString?.count
let result: String = nullableString ?? "Default Value"

if let nullableString {
    print("Hey, \(nullableString)")
} // Doesn't print anything if nullableString is nil.
```

KOTLIN

```kotlin
val length: Int? = nullableString?.length
val result: String = nullableString ?: "Default Value"

// Doesn't print anything if nullableString is null.
nullableString?.let {
    print("Hey, $it")
}
```

Swift vs. Kotlin: Interfaces/Protocols and extensions

SWIFT

```swift
protocol Printable {
    var description: String { get }
}

extension String {
    func reverse() -> String {
        return String(self.reversed())
    }
}
```

KOTLIN

```kotlin
interface Printable {
    var description: String

    fun String.reverse(): String {
        return this.reversed()
    }
}
```

Swift vs. Kotlin: Error handling

SWIFT

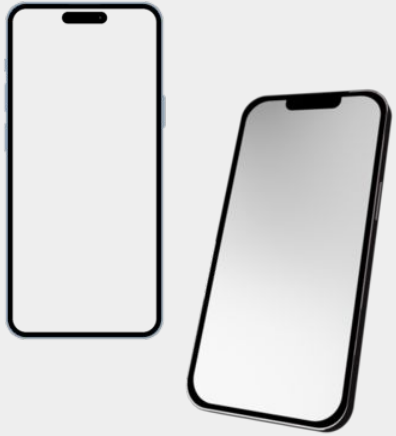
```swift
func checkPassword(_ password: String) throws -> Bool {
    if password == "password" {
        throw PasswordError.obvious
    }
    return true
}

do {
    try checkPassword(userPassword)
    print("That password is good!")
} catch {
    print("You can't use that password.")
}
```

KOTLIN

```kotlin
fun checkPassword(password: String): Boolean {
    if (password == "password") {
        throw PasswordErrorException("Weak password")
    }
    return true
}

try {
    checkPassword(userPassword)
    println("That password is good!")
} catch (e: Exception) {
    println("You can't use that password.")
} finally {
    println("This will always execute.")
}
```



KMP Development

Prerequisites for KMP project (mobile)

- MacOS
- Android Studio + Kotlin Multiplatform Mobile plugin
- Xcode
- *Kdoctor (optional):*

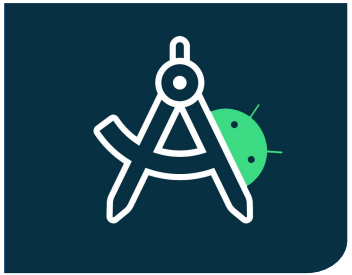
```
Environment diagnose (to see all details, use -v option):
[✓] Operation System
[✓] Java
[✓] Android Studio
[✖] Xcode
  ✖ Current command line tools: /Library/Developer/CommandLineTools
    You have to select command line tools bundled to Xcode
    Command line tools can be configured in Xcode -> Settings -> Locations -> Locations
[!] CocoaPods
  ! CocoaPods configuration is not required, but highly recommended for full-fledged development
  ✖ System ruby is currently used
    CocoaPods is not compatible with system ruby installation on Apple M1 computers.
    Please install ruby via Homebrew, rvm, rbenv or other tool and make it default
    Detailed information: https://stackoverflow.com/questions/64901180/how-to-run-cocoapods-on-apple-silicon-m1/66556339#66556339
  ✖ cocoapods not found
    Get cocoapods from https://guides.cocoapods.org/using/getting-started.html#installation

Conclusion:
  ✖ KDoctor has diagnosed one or more problems while checking your environment.
    Please check the output for problem description and possible solutions.
```

Development in KMP for Mobile

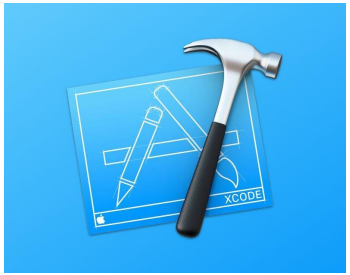
Shared code (Android studio)

- Kotlin
- Gradle
- Kotlin/Native
- *Only* multiplatform libraries
- Kotlin multiplatform gradle plugin



Android (Android studio)

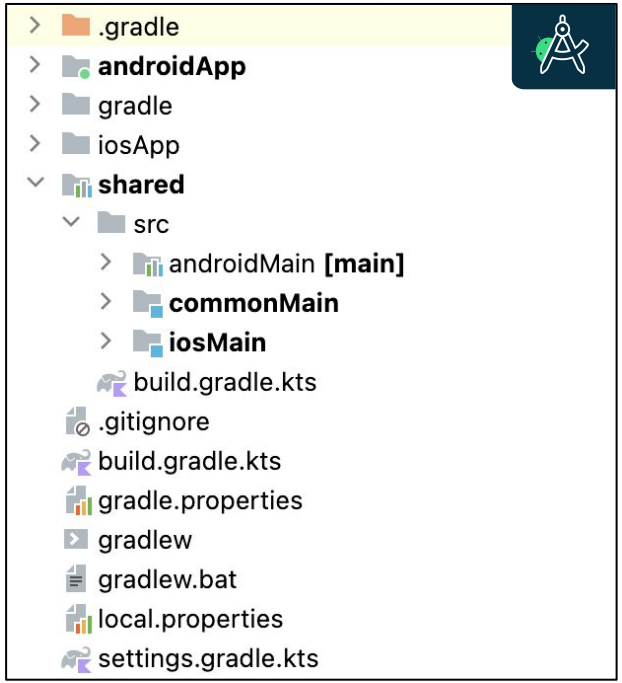
- Kotlin, Java
- Gradle
- XML or Compose
- Android libraries

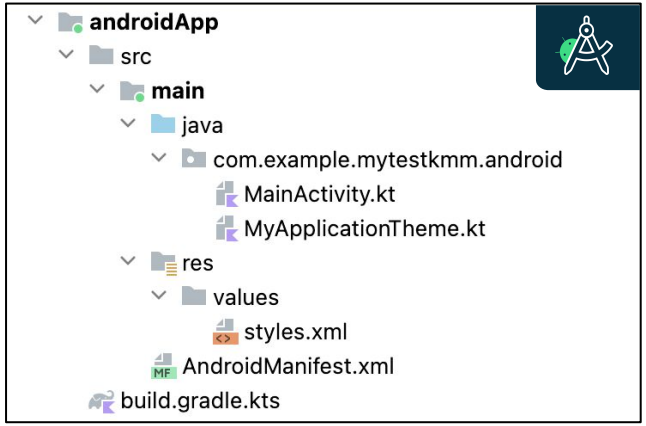


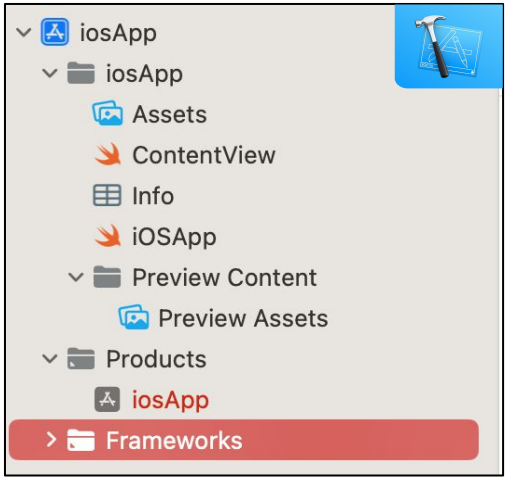
iOS (Xcode)

- Swift, Objective C
- Cocoa Pods, Carthage, Swift Package Manager, …
- SwiftUI or Uikit
- iOS libraries

Structure of KMP project for Mobile







Platform-specific code

- **Expected**
  - Defined in “common” build type
  - What the Kotlin compiler should “expect” during compile
  - Declares the how the function/property/class/interface/enum/annotation looks like

- **Actual**
  - In platform specific build types
  - Needs to be provided for every target platform

Expected declaration is replaced by the **actual** implementation at compile time for each supported platform

Platform-specific code

KOTLIN: expect & actual

```kotlin
expect class PlatformSpecificClass { // Shared code in commonMain
    fun platformSpecificFunction(): String
}

// Android-specific in androidMain
actual class PlatformSpecificClass actual constructor() {
    actual fun platformSpecificFunction(): String {
        return "Android implementation"
    }
}

// iOS-specific in iosMain
actual class PlatformSpecificClass actual constructor() {
    actual fun platformSpecificFunction(): String {
        return "iOS implementation"
    }
}
```

Kotlin Multiplatform libraries

- Kotlin Coroutines
- Ktor (http networking)
- Fuel (http networking)
- Kotlinx.serialization
- Kodein-DI
- Koin
- Room
- SQLDelight
- Jetpack Compose Multiplatform
- And many more …

Kotlin/Native Interoperability

- Kotlin/Native supports two-way interoperability with native programming languages for different operating systems.

- The compiler creates:
  - Apple framework for Swift and Objective-C projects

- Kotlin/Native supports usage of existing libraries directly from Kotlin/Native:
  - Static or dynamic C libraries
  - C, Swift, and Objective-C frameworks

Kotlin/Native interoperability with Swift

- Export from Kotlin to Swift only through Objective-C
- Use special annotations to adjust export of Kotlin declarations to Objective-C/Swift:
  - @ObjCName, @HiddenFromObjC, @ShouldRefineInSwift

- **Issues**:
  - Default arguments not supported
  - Sealed classes are converted to simple classes (non-exhaustive pattern matching)
  - Generics:
    - Possible loss of constraints of generics defined in Kotlin (such as T: Comparable)
    - *Inline classes/Interfaces:* generics can get downcasted to Any?

Helpful tool: *SKIE* (Touchlab, docs)

What's next for Kotlin multiplatform

Fleet

- Currently **deprecated**
- Jetbrains planning new IDE specifically for KMP based on Fleet (2025 roadmap)

Amper:

- Project configuration tool by Jetbrains
- Currently experimental, improvements planned

Direct Kotlin-to-Swift export:

- In preview for now
- Can already try it in Kotlin playground

Compose Multiplatform:

- Stable for Andorid, iOS, desktop and *beta* for web (as of December 2025)

Where to find more

- **Kotlin multiplatform:**
  - https://kotlinlang.org/docs/multiplatform.html#code-sharing-between-platforms
  - https://kotlinlang.org/docs/multiplatform-get-started.html
  - https://www.jetbrains.com/kotlin-multiplatform/
  - https://blog.jetbrains.com/kotlin/category/multiplatform/
  - https://developer.android.com/kotlin/multiplatform
  - https://slack-chats.kotlinlang.org/c/multiplatform
- **Multiplatform libraries:**
  - https://github.com/terrakok/kmp-awesome
  - https://libs.kmp.icerock.dev/
- **Compose Multiplatform:**
  - https://www.jetbrains.com/lp/compose-multiplatform/ https://blog.jetbrains.com/kotlin/2023/11/kotlin-multiplatform-development-roadmap-for-2024/
- **How to write Swift-Friendly API (series of articles):**
  - https://medium.com/better-programming/writing-swift-friendly-kotlin-multiplatform-apis-part-i-1173ec405a20
- **Touchlab tools to simplify KMP development:**
  - https://touchlab.co/blog

Thank you for attention

Feel free to leave feedback about this lesson: